

Operációs rendszerek II.

UNIX bevezetés, történet, belső szerkezet



Várkonyiné Kóczy Annamária

Professzor

Informatika Tanszék

[\(koczya@ujssk.hu\)](mailto:koczya@ujssk.hu)

varkonyi-koczy@uni-obuda.hu

Felhasznált irodalom:

- Kóczy-Kondorosi (szerk.): Operációs rendszerek mérnöki megközelítésben
- Tanenbaum: Modern Operating Systems 2nd. Ed.
- Silberschatz, Galvin, Gagne: Operating System Concepts



13. Bevezetés, történet, belső szerkezet

- Bevezetés
- Történet, trendek
- Felépítés
- A UNIX fejlesztési modellje
- Családfa
- Felépítés

Bevezetés – Miért UNIX?

- ~ 50 év fejlesztési tapasztalat (1969, az egyik legrégebbi OS, jól dokumentált
 - több száz hardver platform (eleinte minden gép egyedi)
 - hardver-szoftver együttes fejlődés
 - számos mai technológia bölcsője
- nyílt (szabadon tanulmányozható) forráskód
 - (nem csak Linux)
- sokrétű fejlesztés – sokrétű alkalmazás
 - a több száz processzoros szerverektől a beágyazott rendszerekig
- eleinte egyszerű („small is beautiful”)
 - (mások: vegyél új hardvert, ha nem fut a szoftverünk)
 - egyszerű, ma is használatos alapelvek, elgondolások

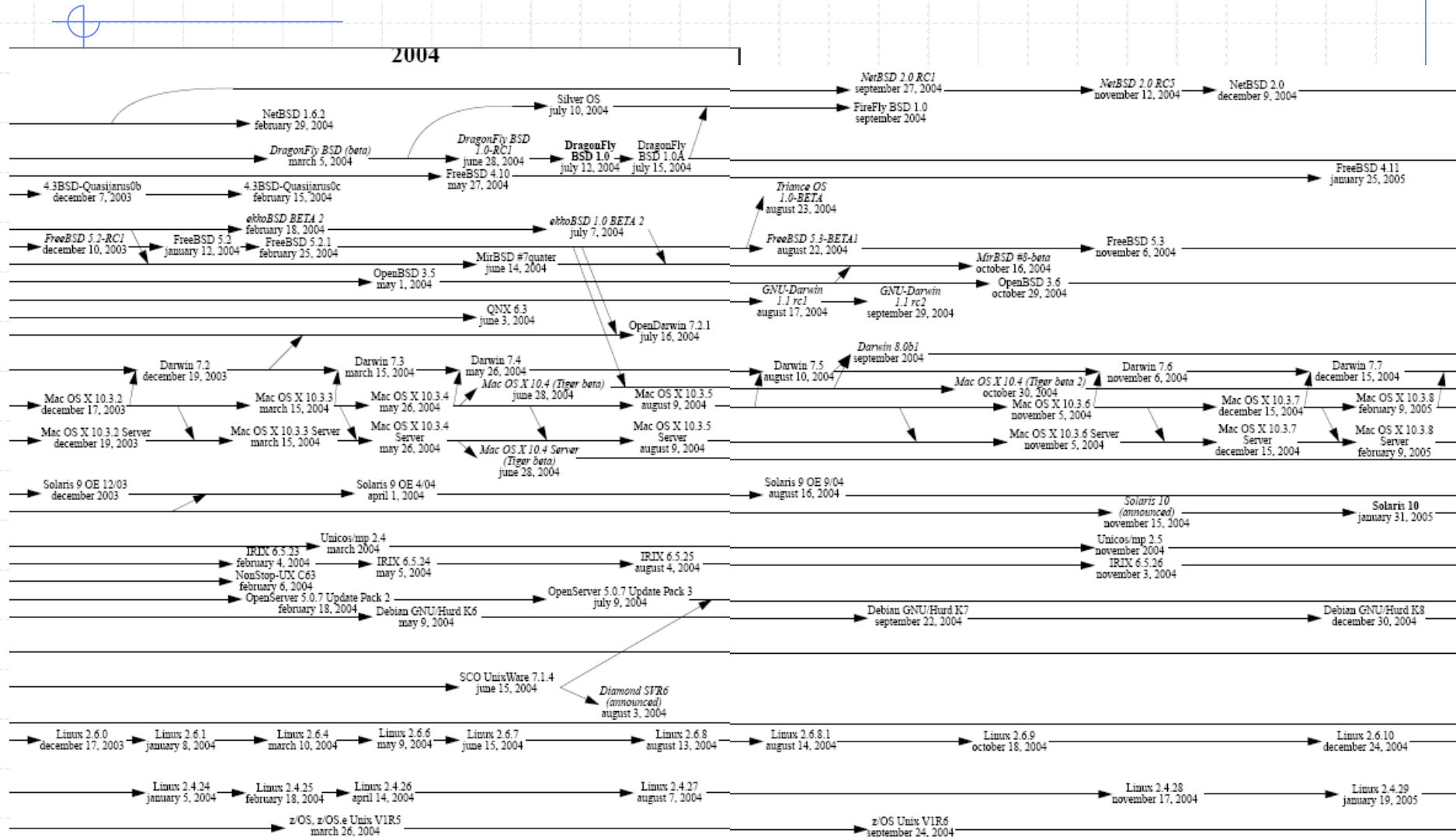
A UNIX története

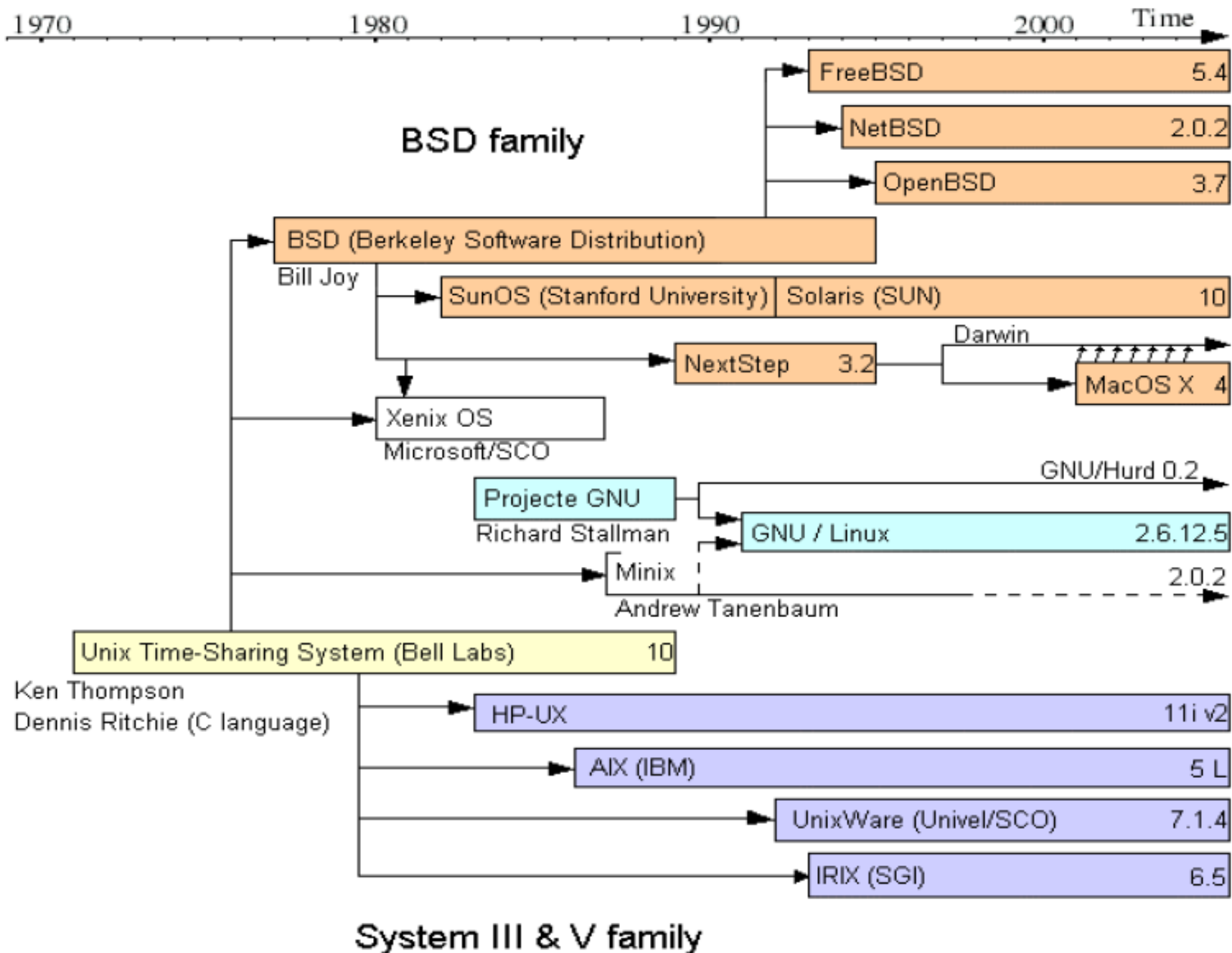
- AT&T Bell Lab, 1969, Ken Thompson, Dennis Ritchie
 - Space Travel játék, drága gépidő, elfekvő PDP-7
 - Lassú futtatórendszer, de gyors játékot akartak
 - Nem volt jó programozási nyelv sem
 - Készítettek egy operációs rendszert (UNIX) és egy programozási nyelvet (C)
 - A UNIX nevet Brian Kernighan ajánlotta
- USA trösztellenes törvény (AT&T)
 - nem kezdhették új üzletágba, emiatt viszonylag olcsó akadémiai licencek
- Sokféle egyedi hardver
- Óriási érdeklődés a forráskódú UNIX (és a C nyelv) iránt
 - 1976-77: Ken Thompson előadása a California-Berkeley egyetemen

A UNIX fejlesztési modell

- Iteratív
- Forráskódú
- Több fejlesztő
- Több hardver platformon párhuzamosan
- Előnyei
 - gyorsan terjed, gyorsan fejlődik
- Hátrányai
 - inkompatibilitás, szakértelmet igényel

A UNIX család





UNIX család és a szabványosítás

- Szabvány: *de jure* és *de facto*
 - de jure: formális testület által formális módon elfogadott specifikáció
 - (de facto: meghatározó súlyú piaci szereplő terméke)
- System V
 - AT&T változat: Sun Solaris, SCO, ...
- BSD
 - Berkeley változat: SunOS, OpenBSD, FreeBSD, ...
- Szabványosítás
 - IEEE POSIX
 - AT&T SVID (pl. SVR4)
 - Open Group: X/Open, Unix95, Unix98, ...
 - ... (desktop, hálózat, stb.)

A UNIX ma ...

- kliens platformon elenyésző, de kitartó
 - Windows 90%, Mac 5%, iOS 2%, Linux 1%, Android 0,5%
 - okostelefon, tablet: átrendezik a piacot (Windows pár éve 94%, iOS 0%)
- szerver platformon jelentős tényező (nem friss adatok)
 - UNIX variánsok 40%, ezen belül a Linux nő, a többi UNIX csökken
 - Windows 50%, stagnál-csökken
 - Netware eltűnő ('99-ben 40%, 2004-ben 10%)
- beágyazott (nem csak ipari)
 - sokféle rendszer, sokféle hardver platform
 - beágyazott Linux, RT-Linux, stb.
 - részben összemosódik a kliens platformmal

(Linux kernel 3D vizualizáció – demó videók)

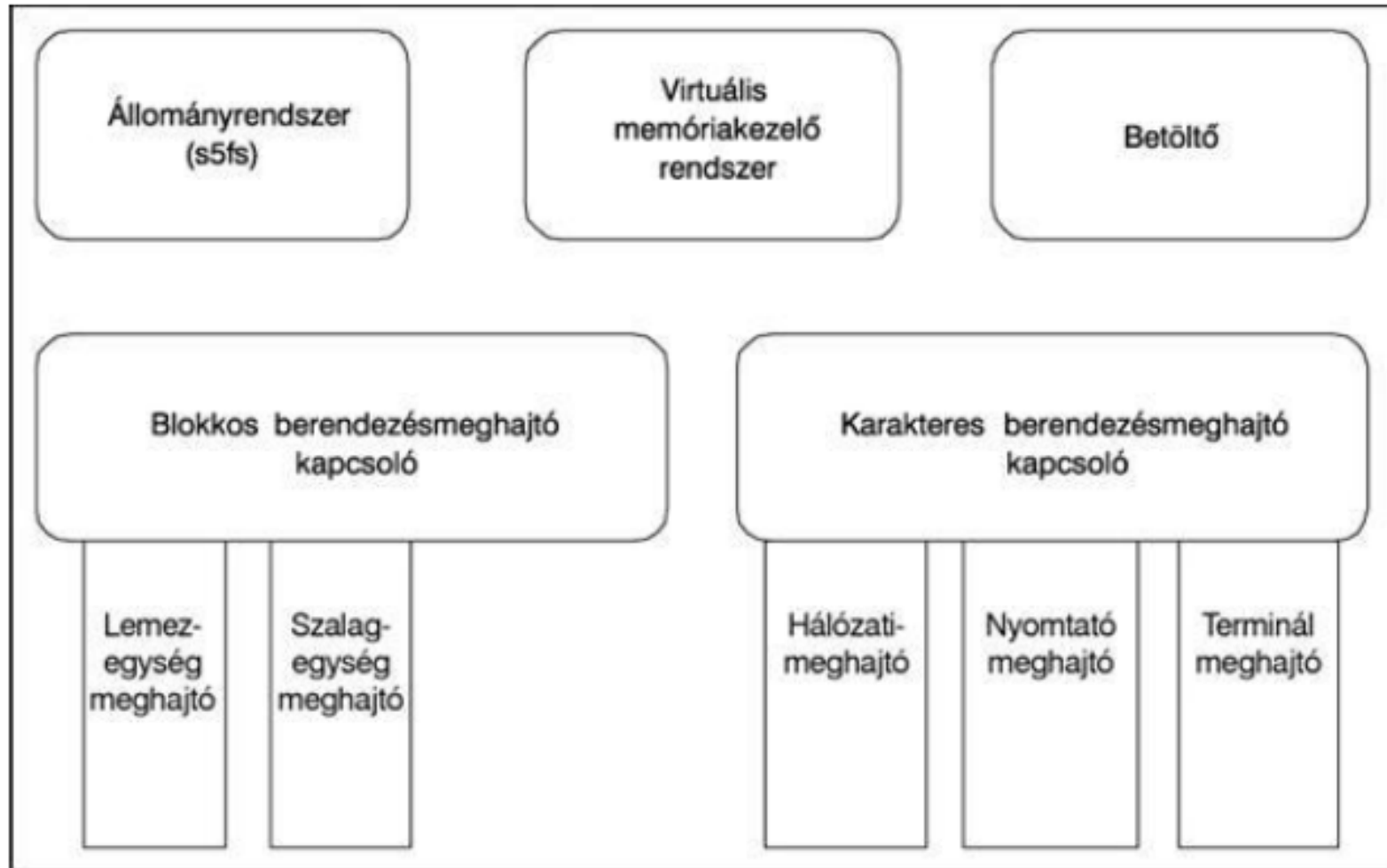
<http://www.pabr.org/kernel3d/kernel3d.html>

További hasznos olvasmányok, érdekes videók:

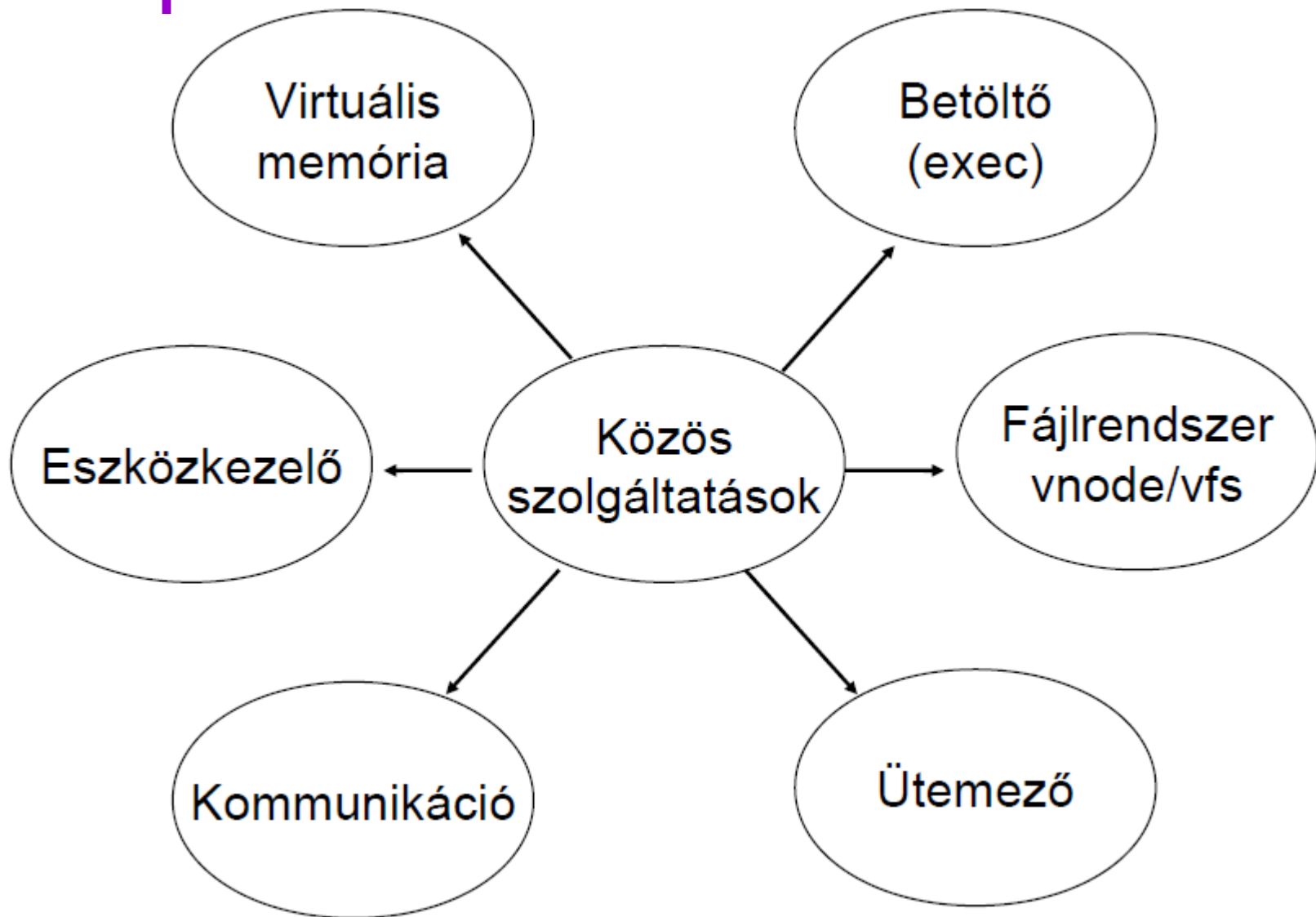
<http://www.jukie.net/bart/blog/linux-kernel-walkthroughs>

http://en.wikiversity.org/wiki/Reading_the_Linux_Kernel_Sources

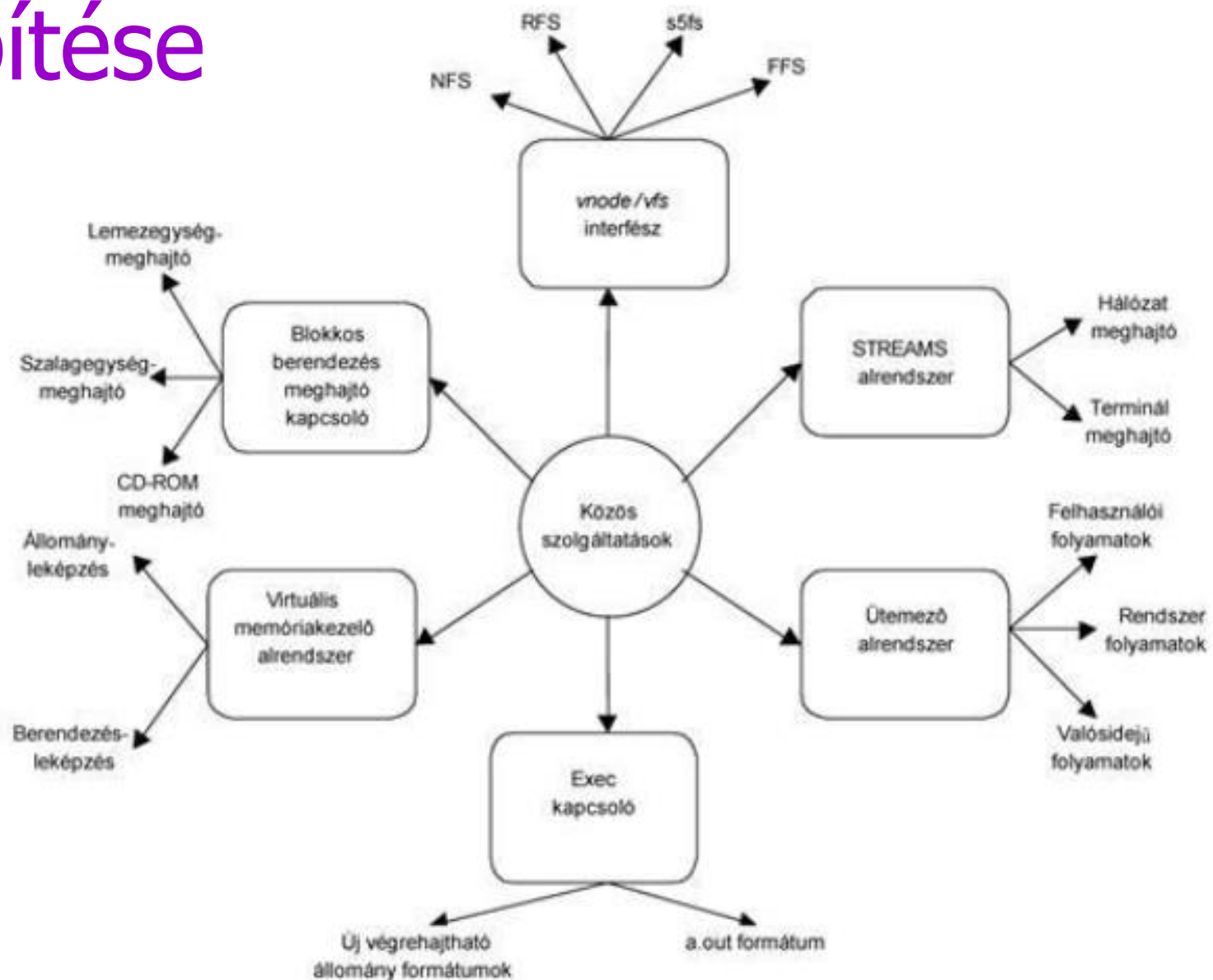
A hagyományos UNIX rendszerek belső szerkezete (~'80-as évekig)



A modern UNIX-ok szerkezeti felépítése



A modern UNIX-ok egy lehetséges felépítése



A UNIX adminisztrátori szemmel

- Karakteres és grafikus konfigurációs felületek (az előbbi preferált)
- Felhasználó azonosítás és hozzáférés-szabályozás
 - alap: jogosultsági hármasság: tulajdonos – csoport – mások
 - haladó (pl.): selinux (pontosan hangolható szerepek és jogosultságok)
- Naplózás, monitorozás és számlázás (kezdetekben \$\$\$\$)
- Eszközkezelés
 - sokféle hardver platform (nem csak PC) és periféria támogatása
- Hálózati szolgáltatások
 - hálózati fájlrendszerek, terheléselosztás, ...
- Vállalati szolgáltatások, virtualizáció, klaszterezés, stb.

A UNIX felhasználói szemmel

- Grafikus és karakteres felhasználói felület (utóbbi látványos)
 - teljes virtuális terminál akár hálózati kapcsolaton át is
 - több párhuzamos felhasználói felület egyenrangú kiszolgálása
- Könyvtárrendszer
- Beépített parancsok (karakteres felületen)
 - tájékozódás: `man`, `info`, `who (am I)` és `w`
 - fájlműveletek: `ls`, `cp`, `mv`, `more` és `less`, `vi`, `zip`, `tar`
 - folyamatkezelés: `ps`, `kill`, `at`, `nice`, `>`, `>>`, `<`, `&`
- Telepített alkalmazások
 - hálózat: `web`, `email`, `chat`, ...
 - szövegszerkesztés (magyarul is)
 - programfejlesztés

Jelentősebb UNIX disztribúciók

- Naprakész katalógus:

<http://distrowatch.com/>

<http://distrowatch.com/dwres.php?resource=major>

- Szerverek üzemeltetésére...
 - RedHat Enterprise Linux és klónjai (CentOS, Scientific Linux)
 - Sun (Oracle) Solaris és változatai (OpenSolaris)
 - IBM AIX, HP-UX
 - SUSE Linux Enterprise Server / openSUSE (Novell), ...
- Klienseken...
 - Ubuntu (Kubuntu, Edubuntu, Mythbuntu, ...)
 - Debian
 - Fedora (RHEL „homokozó”)
 - SUSE Linux Enterprise Desktop / openSUSE (Novell), ...

Folyamatok kezelése - Áttekintés

- Alapismeretek
 - Mi a folyamat? Hogy indul? Hol látszik? Viszonya a kernellel?
 - Kontextusok és végrehajtási módok
- Folyamatok
 - alapadatok
 - állapotok és állapotátmenetek
 - életciklus: létrehozás, futás (ütemezés), leállítás
- Rendszerhívások
 - új folyamat létrehozása
 - új programkód betöltése
- Klasszikus és modern UNIX ütemezők
 - prioritás, ütemezési szintek és működés, preemptivitás
 - modularitás, real-time, interaktív és multimédia folyamatok igényei
 - többprocesszoros rendszerek sajátosságai

Felhasználói ismeretek

- Mi az a folyamat?
 - Feladataink megoldására programokat írunk
 - Egy program futás alatt álló példánya a folyamat (def.)
- Hogyan indulnak a folyamatok?
 - Rendszerinduláskor (rendszerfolyamatok, szolgáltatások)
 - Felhasználó által (elindítja, illetve igény szerinti szolgáltatást kér)
- Hogyan kezeljük a folyamatokat?
 - Listázás (folyamat neve, azonosítója, futtatója, erőforrás adatok, ...)
 - ps, ps -ef, ps axu, ps -u <felhasználó>, ...
 - top, atop és grafikus társaik
 - Vezérlés (leállítás)
 - (felhasználói felületükön)
 - jelzésekkel: CTRL+C, CTRL+Z, kill <JELZÉS> <FOLYAMAT AZONOSÍTÓ>
 - egyebek: pl. prioritás állítás: renice

UNIX folyamatok – kernel alapismeretek

- Folyamatok elkülönítése a kerneltől
 - Végrehajtási mód: különbség a kernel és a folyamat programkódja között
 - Kontextus: kernel és folyamat adatok közötti különbség
- Végrehajtási (futási) módok:
 - Kernel („privilegizált, védett”) mód
 - védett (kernel) tevékenységek végrehajtása
 - Felhasználói („szabad”) mód
 - a folyamat programkódjának végrehajtása
- Végrehajtási környezetek:
 - Kernel (rendszer vagy megszakítás) kontextus
 - a kernel saját feladatainak ellátásához szükséges adatok
 - Folyamat kontextus (virtuális memória-kezelés!)
 - a folyamat futásának adatai (programszöveg, adatok, verem, stb.)
 - a folyamatok kezeléséhez, vezérléséhez szükséges adatok

Végrehajtási mód és környezet



A végrehajtási mód váltása

- A felhasználói mód és a kernel mód között átmenet lebonyolítása
- Jellemzően rendszerhívások meghívása esetén zajlik:
 - a folyamat védett módban végrehajtható tevékenységet szeretne végezni (pl. fájl írása, olvasása, eszközök kezelése, pontos idő lekérdezése, stb.)
 - meghívja a megfelelő rendszerhívást (pl. `open()`, `read()`, `write()`, stb.)
 - Ez a programozók számára klasszikus függvényhívásnak tűnik, de valójában egy speciális szoftver megszakítás, amit a függvényhívást implementáló libc rendszerkönyvtár bonyolít le.
 - a `libc` kiadja a `SYSCALL` utasítást (megszakítást generál)
 - A `SYSCALL` neve hardver függő, pl. `trap`, `syscall`, `sysenter`
 - a CPU kernel (védett) módba vált a megszakítás hatására

A végrehajtási mód váltása (folyt.)

- a CPU kernel (védett) módba vált a megszakítás hatására
- a kernel `SYSCALL` kezelője előkészíti a rendszerhívás végrehajtását
- végrehajtódik a kernel módú eljárás (a rendszerhívás utasításai)
- a kernel visszatér a megszakításból (`iret, sysexit`)
- a CPU felhasználói módba vált a visszatérési utasítás hatására
- a `libc` visszatér a folyamat által meghívott (rendszerhívás) függvényből
- Hardver megszakítások és kivételek (hibák) esetén is kell módváltás

UNIX folyamatok kontextusa részl.

- Folyamat adatok: programszöveg, adatok, vermek, stb.
- Hardver kontextus: cpu, mmu, fpu, stb.
- Adminisztratív adatok (a folyamatok kezeléséhez, vezérléséhez)
 - elsősorban a folyamat futása során szükségesek **u-terület**
A folyamat címtér része
 - hozzáférés-szabályozás adatai
 - rendszerhívások állapotai és adatai
 - nyitott fájl objektumok
 - számlázási és statisztikai adatok, stb.
 - elsősorban a folyamatok kezeléséhez szükségesek **proc struktúra**
A kernel címtér része
 - azonosítók
 - futási állapot és ütemezési adatok
 - memóriakezelési adatok, az u-terület címe, stb.

UNIX folyamatok kontextusa részl. (folyt.)

- Környezeti adatok (a folyamat indításakor megörökölt tulajdonságok)
 - megörökli az őt elindító folyamat környezetét
 - *tulajdonság* = *érték* párok halmaza
 - `set`, `setenv`, `export` parancsokkal a felhasználók is beállíthatják

A folyamatok főbb adminisztrációs adatai

- PID (Process ID): egyedi, a folyamatot azonosító szám
 - PPID: szülő folyamat azonosítója
- A folyamat állapota
 - fut, alszik, stb. (részletesen később)
 - ütemezési információk (prioritás, stb., lásd később)
- Hitelesítők
 - UID: a futtató felhasználó azonosítója (UID=0 a *root* vagy *superuser*)
 - GID: a futtató felhasználó csoport azonosítója
 - valós (real) és effektív (effective) azonosítók (lásd fájlok *setuid* bit)

A folyamatok főbb adminisztrációs adatai (folyt.)

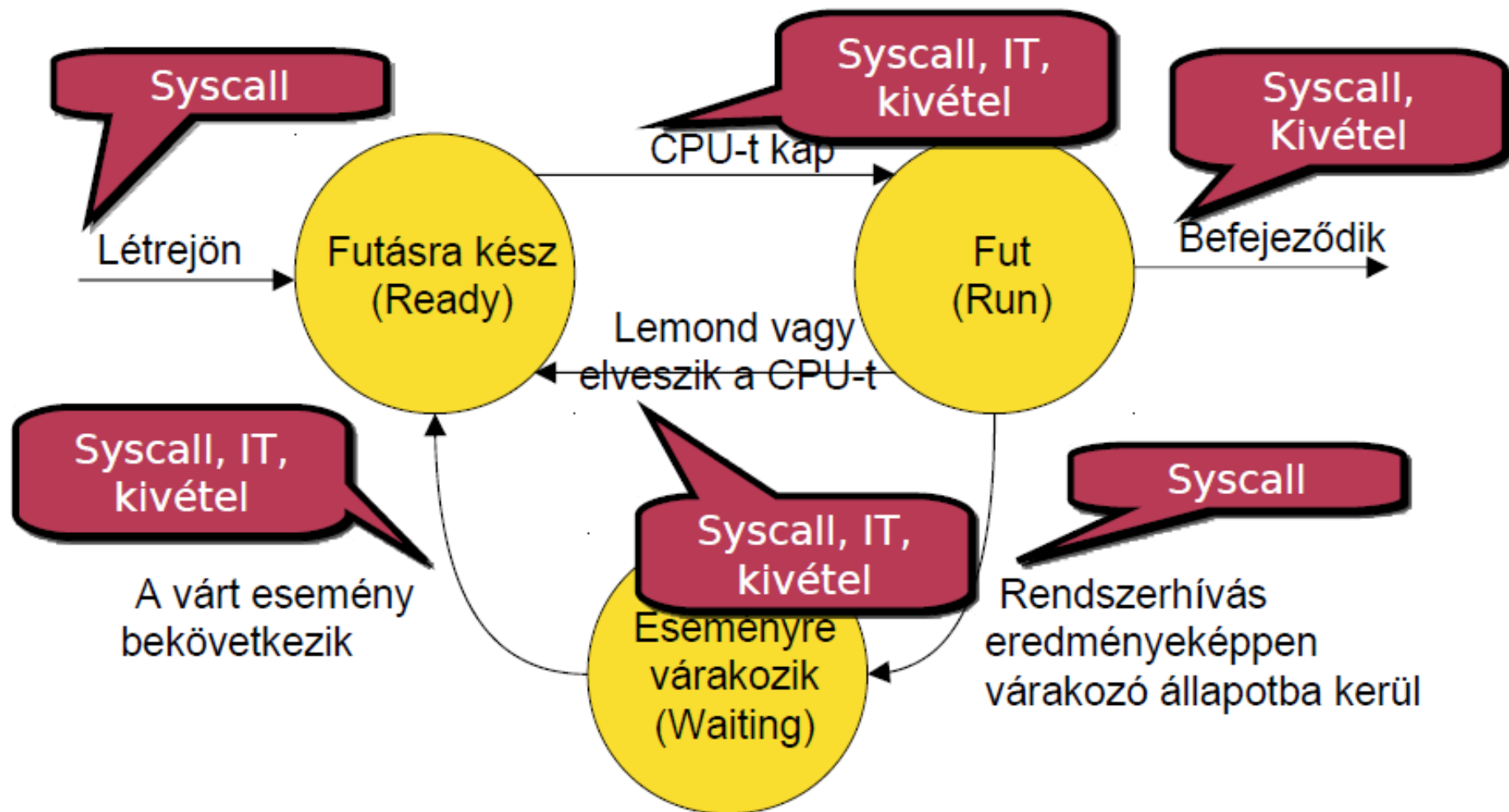
- Memória-kezelési adatok
 - címleképezési térkép
- Kommunikációs adatok
 - fájlleírók, jelzés információk
- Statisztikák
 - erőforrás használat (számlázáshoz)

Folyamatok életciklusa (állapotátmenetek)

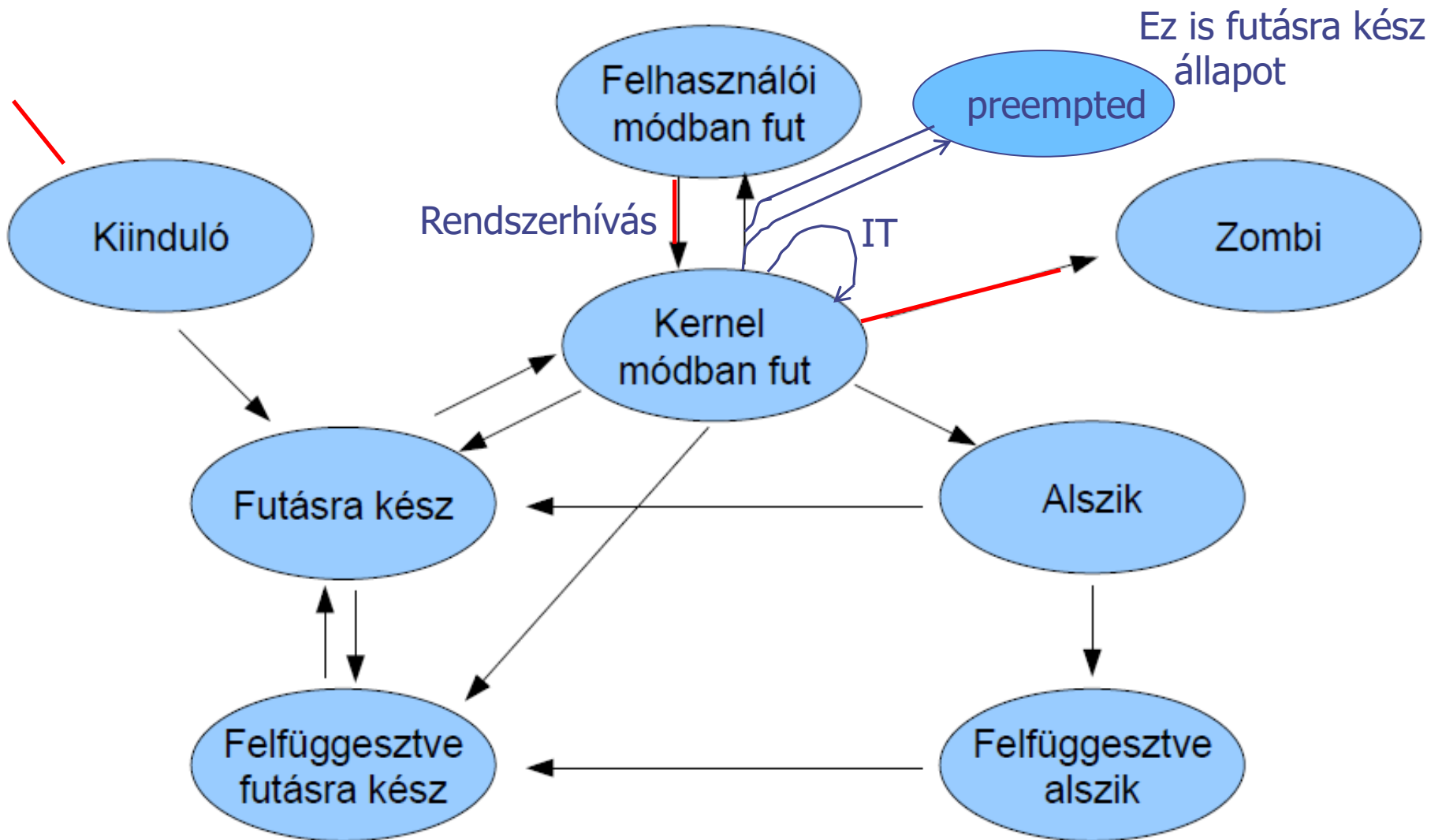
- Létrehozás
 - `fork()`: új folyamat létrehozása
 - `exec()`: új programkód betöltése egy folyamat címtérébe
- Futás és várakozás
 - a futás elindítása (a processzor folyamathoz rendelése)
 - a futás megállítása (a processzor elvétele)
 - várakozás eseményre (önként)
 - felfüggesztés (felhasználó)
 - leállítás (önként vagy a kernel)
 - átütemezés (másik, fontosabb folyamat érkezett)
- Leállítás
 - `exit()` rendszerhívással
 - zombi állapot
 - szülő értesítése
 - gyerekek adoptálása

Feladat állapota (ism.)

- Minden állapotátmenet megszakításra történik!
- A modern operációs rendszerek megszakítás vezéreltek!



UNIX folyamatok állapotai



A folyamatok családfája

- Folyamatot csak egy másik folyamat tud létrehozni
 - minden folyamatnak van szülője és lehetnek gyerekei
 - a szülő változhat (így a gyerekek listája is)
- A `fork()` megadja a szülőnek a gyerek azonosítóját (PID)
- Az ős folyamat (PID 1: `init`, `upstart`, `systemd`, ...)
 - minden folyamat őse
 - a rendszer leállásáig fut
 - örökli az árva folyamatokat
 - figyeli / vezényli bizonyos rendszerfolyamatok meglétét / futását
- A család fontos
 - a szülő értesítést kap a gyerek folyamat leállításáról (nyugtáznia kell)

Összefoglalás

- Folyamatokkal kapcsolatos alapismeretek
 - felhasználói kezelésük: `ps`, `kill`, `nice`
 - futási mód és kontextus
 - rendszerhívások működése
 - metaadataik (u-terület, processz leíró)
- Folyamatok életrajza
 - létrehozás: klasszikusan a `fork()` rendszerhívással
 - új végrehajtandó kód betöltése: `exec()`
 - állapotok (speciális: kétféle futó állapot, felfüggesztett állapotok, zombi)
 - vezérlés alapvetően jelzésekkel
- Folyamatok családfája
 - a `fork()` fát épít, az ős folyamat az `init` (PID 1)
 - a szülők értesülnek a gyerekek leállításáról